Systemsoftware WiSe 2026/27 – Übung 1

Name: _____ Matrikelnummer: _____

Aufgabe 1: Programm, Prozess und Thread

Programm. Ein Programm ist eine *passive*, statische Folge von Anweisungen (plus konstante Daten), die als Datei auf einem Speichermedium liegt, z. B. `Counter.class` oder `count_lines_in_files.py`. Solange es nicht ausgeführt wird, verbraucht es weder CPU-Zeit noch Arbeitsspeicher. Erst wenn das Betriebssystem es lädt und startet, entsteht daraus ein *Prozess*. Dasselbe Programm kann dabei gleichzeitig in mehreren, voneinander unabhängigen Prozessen laufen (z. B. zwei geöffnete Editor-Fenster).

Prozess. Ein Prozess ist ein *Programm in Ausführung*, also eine aktive Instanz eines Programms. Das Betriebssystem ordnet ihm einen eigenen, von anderen Prozessen isolierten virtuellen Adressraum (Code, Daten, Heap, Stack) sowie Ressourcen zu (Prozess-ID, geöffnete Dateien, Rechte, ...); verwaltet wird das im Prozesskontrollblock. Der Prozess ist damit die Einheit der *Ressourcenzuteilung und des Schutzes*. Ausgeführt wird der Code aber von den *Threads* des Prozesses: jeder Prozess besitzt mindestens einen (den Haupt-Thread), kann aber weitere erzeugen.

Thread. Ein Thread ist ein einzelner *Ausführungsstrang innerhalb eines Prozesses* und die Einheit, die der Scheduler einem CPU-Kern zuteilt. Jeder Thread hat seinen eigenen Befehlszähler, eigene Register und einen eigenen Stack, teilt sich aber Adressraum und Ressourcen mit allen anderen Threads *desselben Prozesses*. Dadurch sind Threads „leichtgewichtig“: Erzeugen und Umschalten ist billiger als bei Prozessen, und sie können direkt über den gemeinsamen Speicher kommunizieren. Der Preis dafür ist, dass gemeinsame Daten synchronisiert werden müssen (sonst drohen Race Conditions) und ein fehlerhafter Thread den ganzen Prozess zum Absturz bringen kann.

Bild zur Veranschaulichung: Das Programm ist das Rezept im Kochbuch, der Prozess ist eine Küche, in der nach diesem Rezept gekocht wird (mit eigenen Zutaten und Geräten), und die Threads sind die Köche, die sich in dieser Küche Arbeitsfläche und Zutaten teilen.

Aufgabe 2

a) Grenzen durch mehr Kerne und höhere Taktung

- **Serieller Anteil (Amdahlsches Gesetz):** Fast jedes Programm hat Teile, die nicht parallelisierbar sind (Initialisierung, Einlesen, Zusammenführen der Ergebnisse, Schritte mit Datenabhängigkeiten). Auch mit unendlich vielen Kernen bleibt dieser Teil gleich lang, daher ist der Speedup durch $\frac{1}{1-p}$ begrenzt (p = parallelisierbarer Anteil).
- **Overhead der Parallelisierung:** Threads erzeugen, Arbeit verteilen, synchronisieren (Locks, Warten auf den langsamsten Thread), Kontextwechsel und Cache-Kohärenz-Verkehr kosten Zeit. Ab einer gewissen Kernanzahl überwiegt dieser Mehraufwand den Gewinn, das Programm kann sogar wieder langsamer werden.
- **Gemeinsam genutzte Ressourcen:** Alle Kerne teilen sich Speicherbandbreite, Caches, Bus und Ein-/Ausgabe (Festplatte, Netzwerk). Ist ein Prozess speicher- oder I/O-gebunden,

wartet die CPU ohnehin die meiste Zeit; dann helfen weder mehr Kerne noch ein höherer Takt („Memory Wall“).

- **Physikalische Grenzen der Taktrate:** Die Leistungsaufnahme wächst etwa mit $P \sim C \cdot V^2 \cdot f$, und höhere Frequenzen benötigen höhere Spannung. Wärmeentwicklung und Kühlbarkeit („Power Wall“) sowie Signallaufzeiten begrenzen daher den Takt; deshalb stagnieren Taktraten seit Mitte der 2000er bei wenigen GHz. Viele aktive Kerne senken zudem den Turbo-Takt der einzelnen Kerne.

b) Speedup nach Amdahl

Mit dem parallelisierbaren Anteil p und der Kernanzahl n gilt

$$S(n) = \frac{1}{(1-p) + \frac{p}{n}}.$$

Rechenweg (Beispiel $p = 0{,}5$, $n = 4$):

$$S(4) = \frac{1}{(1-0{,}5) + \frac{0{,}5}{4}} = \frac{1}{0{,}5 + 0{,}125} = \frac{1}{0{,}625} = 1{,}6$$

Zweites Beispiel ($p = 0{,}75$, $n = 16$):

$$S(16) = \frac{1}{0{,}25 + \frac{0{,}75}{16}} = \frac{1}{0{,}25 + 0{,}046875} = \frac{1}{0{,}296875} \approx 3{,}368$$

p	$n = 2$	$n = 4$	$n = 8$	$n = 16$	Grenze $n \to \infty$: $\frac{1}{1-p}$
25 %	1,143	1,231	1,280	1,306	1,333
50 %	1,333	1,600	1,778	1,882	2,000
75 %	1,600	2,286	2,909	3,368	4,000

Man sieht deutlich den abnehmenden Grenznutzen: Bei $p = 25\,\%$ bringen 16 Kerne kaum mehr als 2 Kerne, und selbst bei $p = 75\,\%$ erreicht man mit 16 Kernen nur einen Speedup von etwa 3,4, weil die verbleibenden 25 % seriell laufen.

Aufgabe 3: Line Counter

Testumgebung

- Linux, 2 CPU-Kerne (Intel Xeon, 2,1 GHz), OpenJDK 21, Python 3.13
- Testdaten: 43 539 Python-Quelldateien (`.py`), insgesamt 638 MB und 18 654 579 Zeilen; Aufruf jeweils mit Regex `.*\.py`
- Jede Konfiguration wurde 5-mal gemessen, angegeben ist der Median. Die Dateien lagen dabei im Page-Cache des Betriebssystems (warmer Cache).

3.1 Originalprogramm

Beide Originalversionen liefern dasselbe Ergebnis (18 654 579 Zeilen). Laufzeit: Java `Counter` ca. 2220 ms, Python `count_lines_in_files.py` ca. 2005 ms.

Auffälligkeit: Das Java-Programm bricht mit einer `MalformedInputException` ab, sobald eine Datei kein gültiges UTF-8 enthält, weil `Files.lines()` standardmäßig UTF-8 dekodiert. Die Python-Version ignoriert solche Fehler (`errors='ignore'`). Für die Messungen wurden die 3 betroffenen Dateien aus den Testdaten entfernt.

3.2 Parallelisierte Variante (Java, `ParallelCounter.java`)

Vorgehen:

1. Der Verzeichnisbaum wird (sequenziell) mit `Files.walk` durchlaufen, alle passenden Dateien landen in einer Liste.
2. Ein `ExecutorService` mit fester Thread-Anzahl T (Kommandozeilenparameter) wird erzeugt. Thread i bearbeitet die Dateien mit Index $i, i+T, i+2T, \ldots$ Diese verschränkte Aufteilung verteilt große und kleine Dateien gleichmäßiger als zusammenhängende Blöcke.
3. Jeder Thread summiert in einer *lokalen* Variable und gibt seine Teilsumme als `Future<Long>` zurück; der Haupt-Thread addiert die Teilsummen. Weil kein Thread in gemeinsame Variablen schreibt, ist keine Synchronisation nötig und es gibt keine Race Condition.

```
ExecutorService pool = Executors.newFixedThreadPool(threads);
List<Future<Long>> results = new ArrayList<>();
for (int t = 0; t < threads; t++) {
    final int id = t;
    results.add(pool.submit(() -> {
        long sum = 0;
        for (int i = id; i < files.size(); i += threads)
            sum += countLines(files.get(i).toString());
        return sum;
    }));
}
long total = 0;
for (Future<Long> f : results) total += f.get(); // wartet auf jeden Thread
pool.shutdown();
```

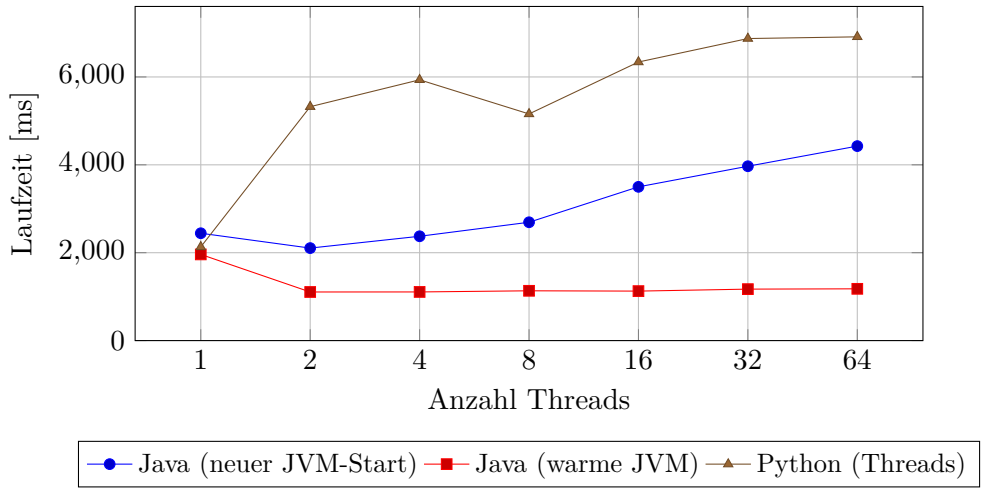
Aufruf:

```
java ParallelCounter <Ordner> <Regex> [Threads]
```

Zusätzlich misst `Benchmark.java` alle Thread-Anzahlen nacheinander in einer JVM nach Aufwärmläufen. Zum Vergleich gibt es eine analoge Python-Version (`count_lines_parallel.py`, `ThreadPoolExecutor`).

3.3 Experimente und Beobachtungen

Threads	seq. Original	1	2	4	8	16	32	64
Java, je ein JVM-Start [ms]	2220	2443	2105	2374	2693	3499	3968	4426
Java, warme JVM [ms]	–	1961	1106	1107	1133	1126	1171	1178
Speedup ggü. 1 Thread	–	1,00	1,77	1,77	1,73	1,74	1,68	1,67
Python, Threads [ms]	2005	2132	5323	5935	5158	6340	6874	6913



- **Korrektheit:** In allen Läufen und bei jeder Thread-Anzahl wurde dieselbe Zeilenzahl (18 654 579) ermittelt.
- **Speedup bis zur Kernanzahl:** In der aufgewärmten JVM halbiert sich die Laufzeit beinahe beim Wechsel von 1 auf 2 Threads (Speedup 1,77). Mehr Threads als Kerne (hier 2) bringen keinen weiteren Gewinn, da nie mehr als 2 Threads gleichzeitig rechnen können. Ab 32/64 Threads wird es durch Thread-Erzeugung, Kontextwechsel und Scheduling-Overhead wieder etwas langsamer.
- **Bezug zu Amdahl:** Das Durchlaufen des Verzeichnisbaums ist in meiner Lösung sequenziell und dauert ca. 150 ms von ca. 1960 ms, also ist $p \approx 0{,}92$. Für $n = 2$ ergibt sich $S = 1/(0{,}08 + 0{,}92/2) \approx 1{,}86$, was gut zum gemessenen Wert 1,77 passt (Rest: Overhead, gemeinsame Speicherbandbreite).
- **Einfluss des JIT-Compilers:** Wird für jede Messung eine neue JVM gestartet, ist der Gewinn kaum sichtbar (2 Threads: 2105 ms statt 2443 ms) und viele Threads verschlechtern die Laufzeit deutlich. Der Grund: In einer frischen JVM laufen anfangs der Interpreter sowie JIT-Compiler- und Garbage-Collector-Threads, die auf nur 2 Kernen mit den Arbeits-Threads um CPU-Zeit konkurrieren. Mit 1 Thread ist die parallele Variante zudem etwas langsamer als das Original, da zusätzlich die Dateiliste aufgebaut und ein Thread-Pool verwaltet werden muss.
- **Python und der GIL:** In Python werden die Threads sogar um den Faktor 2,5–3,2 *langsamer*. Der Global Interpreter Lock (GIL) erlaubt immer nur einem Thread, Python-Bytecode auszuführen. Das Zeilenzählen ist hier (warmer Cache) CPU-gebunden (Dekodieren und Iterieren über Zeilen), daher findet keine echte Parallelität statt; stattdessen kostet das ständige Weiterreichen des GIL zwischen den Kernen zusätzlich Zeit. Echte Parallelität bekäme man in Python mit `multiprocessing` (mehrere Prozesse mit je eigenem GIL) oder der GIL-freien Python-Variante (ab 3.13 experimentell).
- **Fazit:** Für CPU-gebundene Arbeit ist eine Thread-Anzahl gleich der Anzahl der Kerne optimal. Wären die Dateien nicht im Cache (I/O-gebunden, z. B. HDD), könnten etwas mehr Threads helfen, Wartezeiten zu überlappen; die Festplatte wird dann aber selbst zum Engpass.